Windows Autopilot — Government (GCC High)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — Windows Autopilot (Government / GCC High) 5
The situation . . . 5
The complication . . . 5
The resolution . . . 5
The outcome . . . 6
Bottom line . . . 6
Solution Overview — Windows Autopilot (Government / GCC High) 6
What this solution delivers . . . 6
Commercial vs GCC High — what's different . . . 6
How the supported model works in GCC High (plain language) . . . 8
The standard deployment model (all solutions) . . . 9
Named objects this solution creates (used verbatim throughout) . . . 9
Who this is for . . . 10
Sources . . . 10
Requirements — Windows Autopilot (Government / GCC High) 10
R1. Genuine GCC High (or DoD) tenant — not Commercial/GCC . . . 10
R2. US Government licensing (Gov SKUs) . . . 11
R3. Administrative roles for device preparation . . . 11
R4. Supported Windows 11 build for device preparation . . . 11
R5. Device not already registered as a classic Autopilot device . . . 11
R6. Intune Provisioning Client service principal present . . . 11
R7. Users may join devices to Microsoft Entra + automatic enrollment enabled . . . 12
R8. Microsoft Graph PowerShell SDK with US Gov environment . . . 12
R9. Sovereign endpoints reachable; no SSL inspection on the MDM service . . . 12
R10. (Conditional) Corporate identifiers, only if blocking personal devices . . . 12
Assess gate (pass/fail summary) . . . 13
Sources . . . 13
Global Requirements — Intune Solutions (category-level) 13
G1. Microsoft Entra ID tenant . . . 13
G2. Microsoft Intune subscription . . . 14
G3. Administrative roles . . . 14
G4. Device enrollment in Intune . . . 14
G5. Pilot security group . . . 14
G6. Supported Windows baseline . . . 14
G7. Network access to Microsoft cloud endpoints . . . 14

Implementation Guide — Windows Autopilot (Government / GCC High) 14
Phase A — Configure (build the objects) . . . 15
Phase B — Pilot (provision real devices) . . . 19
Sources . . . 19
Testing & Validation — Windows Autopilot (Government / GCC High) 19
T1 — Configuration objects exist and are correct (pre-pilot) . . . 20
T2 — Tenant is genuinely GCC High (not Commercial) . . . 20
T3 — Endpoint reachability from the provisioning network . . . 20
T4 — End-to-end pilot provisioning (the core test) . . . 21
T5 — Deployment monitoring reflects the run . . . 21
T6 — Servicing and policy land (post-provisioning) . . . 21
Validate gate (pass/fail summary) . . . 21
Sources . . . 22
Troubleshooting — Windows Autopilot (Government / GCC High) 22
1. OOBE completes but the device group is empty and nothing installed . . . 22
2. The device group is Dynamic, not Assigned . . . 22
3. "I don't see Windows Autopilot / Deployment Profiles / hardware-hash import" . . . 23
4. Scripts return nothing / see the "wrong" tenant . . . 23
5. Provisioning fails at sign-in / can't reach services . . . 23
6. Device won't provision — unsupported Windows build . . . 23
7. Self-deploying / pre-provisioning / Hybrid join / device-rename options are missing . . . 23
8. Too many apps during OOBE / low provisioning success rate . . . 23
9. Windows Hello for Business PIN reset doesn't work . . . 24
10. Update/servicing policies don't appear or don't apply . . . 24
11. Compliance policy always evaluates non-compliant on an attestation rule . . . 24
12. Certificate provisioning expected via Cloud PKI . . . 24
13. Remote assistance for a stuck device isn't available . . . 24
14. Device was previously a classic Autopilot device elsewhere . . . 24
15. RBAC admin can't configure device preparation . . . 24
Escalation checklist (attach to any support case) . . . 25
Sources . . . 25
Best Practices — Windows Autopilot (Government / GCC High) 25
1. Design for the supported path only — never for classic Autopilot . . . 25
2. Treat the Provisioning Client owner as the linchpin . . . 26
3. Keep the OOBE app/script list short . . . 26
4. Service Windows through update rings, not Autopatch . . . 26

5. Build compliance on locally evaluated signals . . . 26
6. Plan the gaps that touch day-2 support . . . 26
7. Get the network right before the first device . . . 26
8. Use the scripts the way they are designed . . . 27
9. Respect the gates — pilot before you expand . . . 27
10. Keep a "verify for GCC High" watchlist . . . 27
Sources . . . 27
Deliverables Summary — Windows Autopilot (Government / GCC High) 27
Documents (in `source/`) . . . 28
PowerShell scripts (in `scripts/`) . . . 28
Named objects you will have built . . . 29
Definition of done . . . 29
The one thing to remember . . . 29
Sources . . . 29
Appendix A — PowerShell Scripts 31

Executive Summary — Windows Autopilot (Government / GCC High)

The situation

Your organization operates on **GCC High** (or DoD) — the US sovereign cloud required for regulated government and defense-industrial-base workloads (CMMC, ITAR, DFARS). You want the outcome enterprises get from **Windows Autopilot**: a new Windows 11 device is powered on, the user signs in, and it configures itself into a managed, compliant, ready-to-work state with **no imaging and no technician touch**.

The complication

GCC High is not the Commercial portal with a different sign-in. It is a physically separate cloud (Azure Government, IL4/IL5) with `*.us` admin portals, sovereign service endpoints, and — decisively — **a reduced feature set**. Most consequentially for this workload:

> ***Classic Windows Autopilot is not available in GCC High or DoD, and Microsoft has no plan to bring it.*** *Teams that lift-and-shift a Commercial Autopilot runbook build against a hardware-hash import and a "Deployment Profiles" blade* ***that do not exist here*** *— wasting a project cycle before discovering the gap.*

Additional gaps that quietly break Commercial designs: **self-deploying and pre-provisioning (white-glove) modes and Hybrid Entra join are not available**; **Windows Autopatch and the newer update policies are unavailable**; and **Microsoft Store for Business, Cloud PKI, Device Health Attestation, DFCI, and Remote Help are unavailable**.

The resolution

This solution delivers Autopilot on the **one supported path**: **Windows Autopilot device preparation** (user-driven, Microsoft Entra join), which **is** supported in GCC High/DoD. It provides:

- The correct `*.us` **portal navigation** end to end, with every Commercial-vs-GCC-High

difference called out at the step where it matters.

- A named, reusable object set (provisioning users group, **assigned** device group owned by

the **Intune Provisioning Client** service principal, device preparation profile, enrollment status page, least-privilege RBAC role, update ring, compliance policy).

- **Five PowerShell scripts** that connect to **US Gov Graph** (`-Environment USGov`) for

read-only discovery, validation, and reporting, plus **safe** remediation and scope targeting that default to `-WhatIf` (preview before change).

- A **cited GCC High parity reference** separating doc-confirmed facts from items flagged

"verify for GCC High," so nothing is taken on faith.

- Testing and troubleshooting with **pass/fail gates** and the GCC-High-specific failure modes

(most notably the empty-device-group symptom when the provisioning service principal is not the group owner).

The outcome

Windows Autopilot stood up **correctly on GCC High** — right portals, right supported technology (device preparation), parity gaps handled, servicing designed around update rings rather than unavailable Autopatch — and **proven on a pilot** before you scale, using the standard **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model.

Bottom line

Self-guided, tested, sovereign-correct — at a fraction of the cost of a consulting engagement, and without the failed first attempt that comes from running Commercial Autopilot steps on a government tenant.

All feature-availability claims are drawn from Microsoft US Government documentation; see `gcc-high-parity-notes.md` for the full cited list and any "verify for GCC High" flags.

Solution Overview — Windows Autopilot (Government / GCC High)

What this solution delivers

This is the **Government (GCC High) edition** of Windows Autopilot. The business goal is the one every organization wants from Autopilot: a new Windows 11 device ships from the OEM, an end user powers it on, signs in with their organizational account, and — with **no imaging, no manual domain join, and no technician touch** — the device provisions itself into a managed, compliant, ready-to-work state, **Microsoft Entra joined** and Intune-enrolled.

What is different here is **where** and **how** you build it. GCC High is a physically separate, US-sovereign cloud (built on Azure Government, IL4/IL5) with its own admin portals (`*.us`), its own service endpoints, and — critically — **a different feature set** than Commercial. Following a Commercial Autopilot runbook on a GCC High tenant does not error politely; it sends you looking for a portal blade that was never provisioned. This edition is authored so a government tenant is configured **correctly**, on the supported technology, with every Commercial-vs-GCC-High gap called out at the step where it bites.

> ***The single most important fact in this solution — read it before anything else:*** *Classic **Windows Autopilot is NOT available in GCC High or DoD**, and Microsoft has **no plan to bring it** (Microsoft, *Intune Government Service overview*, table "Intune features not available in GCC High and DoD" lists **Windows Autopilot** as **n/a**). The **supported** zero-touch path in GCC High is **Windows Autopilot device preparation** — user-driven, Microsoft Entra join. **This entire solution is built on device preparation.** Every place a Commercial guide says "Autopilot," read "Autopilot **device preparation**."*

Commercial vs GCC High — what's different

This is the section to read before you touch anything. Every difference below is doc-confirmed against Microsoft US Gov documentation unless explicitly flagged *"verify for GCC High."*

1. The Autopilot technology itself is different (the pivot)

Capability	Commercial	GCC High / DoD
Classic Windows Autopilot (deployment profiles, hardware-hash registration, "Windows Autopilot devices" import, deployment groups)	Supported	**Not available. No plan to support.** (*Intune Government Service overview*)
Windows Autopilot device preparation (user-driven, Entra join)	Supported	**Supported** (*Intune Government Service overview*; *Autopilot device preparation overview*)
Self-deploying mode (kiosk/shared)	Supported (classic Autopilot)	**Not yet supported — planned** (*Intune Government Service overview*)
Pre-provisioning (white-glove)	Supported (classic Autopilot)	**Not yet supported — planned**
Hybrid Microsoft Entra join Autopilot	Supported (classic Autopilot)	**Not available** — device preparation is **Entra join only** (*Autopilot device preparation overview*)
Rename device during OOBE based on org structure	Supported	**Not yet supported — planned**
DFCI management (UEFI/BIOS lock via Autopilot)	Supported	**Not available — planned**

Consequence for design: you cannot design around a hardware-hash CSV, a "Deployment Profiles" blade, a "Windows Autopilot devices" import, self-deploying kiosks, pre-provisioning (white-glove), Hybrid Entra join, or Autopilot device renaming. This solution uses **device preparation profiles** with **Entra Join + Enrollment Time Grouping** instead. If you are coming from a Commercial Autopilot background, this is the mental model shift: **there is no hardware hash and no device registration step** in the user-driven device-preparation flow.

2. The portals are different (`*.us`, not `*.com`)

Purpose	Commercial URL	GCC High URL
Intune admin center	`intune.microsoft.com`	`https://intune.microsoft.us`
Entra admin center	`entra.microsoft.com`	`https://entra.microsoft.us` *(verify for GCC High — see parity notes)*
Azure portal	`portal.azure.com`	`https://portal.azure.us`
Microsoft 365 admin	`admin.microsoft.com`	`https://portal.office365.us`
Sign-in / identity (Evo STS)	`login.microsoftonline.com`	`https://login.microsoftonline.us`
Company Portal service	`portal.manage.microsoft.com`	`https://portal.manage.microsoft.us`
Microsoft Graph	`graph.microsoft.com`	`https://graph.microsoft.us`

(Portal and endpoint URLs: Microsoft, *Network endpoints for US government deployments*; *Microsoft Graph national cloud deployments*.)

3. Servicing and update capabilities are reduced

Several servicing features a Commercial Autopilot rollout leans on **do not exist in GCC High**:

- **Windows Autopatch** — not available.
- **Feature updates, Quality updates, Expedite updates, Driver updates for Windows in Intune** — not available.
- **Windows Subscription Activation** (auto step-up Pro → Enterprise) — not available.
- **Delivery Optimization for Win32 apps** — not available.

Consequence: in GCC High you service Windows through **Update rings for Windows 10 and later** (which *are* available). Do not build a rollout that assumes Autopatch or the newer per-update-class policies.

4. Other feature gaps that touch provisioning

- **Microsoft Store for Business** — not available (use Win32/line-of-business apps).
- **Remote Help** — not available (plan an alternative remote-assist tool for post-provisioning support).
- **Cloud PKI** — not available in GCC High (use SCEP/PKCS with your own PKI/NDES).
- **Device Health Attestation (DHA)** — not available (do not build compliance rules that depend on attestation).
- **Windows Hello for Business** — the WHfB policy path works, but the **Microsoft hosted PIN reset service is not available in Azure Government**; PIN recovery must be configured differently. *(See parity notes; verify for GCC High for your Windows build.)*

5. What is the same

The core experience genuinely transfers: **Entra Join**, **automatic MDM enrollment**, an **assigned device group** with the **Intune Provisioning Client** service principal as its owner (Enrollment Time Grouping), **app and PowerShell-script delivery during OOBE**, **configuration and compliance policies**, an **Enrollment Status Page** progress experience, and **near-real-time deployment monitoring**. The muscle memory transfers; the portals and the servicing model do not.

How the supported model works in GCC High (plain language)

1. You publish a **Windows Autopilot device preparation** profile in the GCC High Intune admin center (`https://intune.microsoft.us`).
1. The profile points at an **assigned** Microsoft Entra security group (the *device group*) whose **owner** is the **Intune Provisioning Client** service principal. That ownership is the mechanism — called *Enrollment Time Grouping* — that lets Intune drop each freshly enrolled device into the group **at enrollment time**, so the selected apps, scripts, and policies deploy immediately.
1. A user in your **provisioning user group** is allowed to Entra-join devices and is

Intune-licensed.

1. The user powers on a supported Windows 11 device, connects to a network that reaches the

sovereign endpoints, signs in with their GCC High account, and the device Entra-joins, MDM-enrolls, installs the selected apps/scripts, applies policy, and lands the user on the desktop with a progress percentage the whole way.

No serial-number hardware-hash upload is required for the user-driven flow — that is a *classic Autopilot* concept that does not apply here. Optionally, **corporate identifiers** can be pre-uploaded if you use enrollment restrictions to block personal devices.

The standard deployment model (all solutions)

This solution follows the standard, gated model. **Do not skip a gate.**

Assess → **Configure** → **Pilot** → **Validate** → **Expand.**

- **Assess** — confirm every prerequisite in `02-requirements.md` (GCC High tenant, US Gov

licensing, roles, sovereign portal access, supported Windows 11 build, endpoints allowed through filtering). *Gate:* every requirement verifies before you configure.

- **Configure** — build the named objects in `03-implementation.md` using the GCC High portals.
- **Pilot** — provision a small set of real devices for the pilot group.
- **Validate** — run the tests in `05-testing.md`; every gate must pass.
- **Expand** — widen scope group by group using `WAPG-Set-ScopeTargeting.ps1` (default `-WhatIf`).

Named objects this solution creates (used verbatim throughout)

Reuse these exact names in every step and script so the pieces line up:

Object	Exact name
Provisioning user group (allowed to Entra-join, Intune-licensed)	`WAPG-Provisioning-Users`
Device security group (**Assigned**; owner = Intune Provisioning Client)	`WAPG-Autopilot-Devices`
Pilot device/user group	`WAPG-Pilot`
Autopilot **device preparation** profile	`WAPG-DevicePrep-UserDriven-EntraJoin`
Enrollment Status Page profile	`WAPG-ESP-Provisioning`
Custom Intune RBAC role	`WAPG Device Preparation Administrator`
Windows update ring	`WAPG-UpdateRing-Pilot`
Compliance policy	`WAPG-Compliance-Win11`
Deployment scope tag	`WAPG`

Who this is for

Novice-to-intermediate administrators standing up Windows Autopilot on a **GCC High** (or DoD) tenant who need the correct sovereign portals, the **supported** provisioning technology (device preparation), and the Commercial-vs-GCC-High gaps called out — instead of a Commercial runbook that silently fails.

Sources

- Microsoft, *Microsoft Intune Government Service overview* (GCC High / DoD feature availability: classic Autopilot **not available**; device preparation **supported**; self-deploying/pre-provisioning/OOBE-rename **planned**).
- Microsoft, *Overview of Windows Autopilot device preparation* (GCCH/DoD support, Entra-join-only, user-driven, Enrollment Time Grouping, Autopilot precedence).
- Microsoft, *Windows Autopilot device preparation requirements* (software, networking, licensing, configuration, RBAC).
- Microsoft, *Network endpoints for US government deployments* (portal and service URLs).
- Microsoft, *Microsoft Graph national cloud deployments* (`-Environment USGov`).

Requirements — Windows Autopilot (Government / GCC High)

This is the **Assess** gate. Confirm **every** requirement below **before** you configure anything in `03-implementation.md`. Each entry gives **Description · Why · Verify · Implement-if-missing**. The category-level prerequisites (Entra tenant, Intune subscription, admin roles, enrollment, pilot group, Windows baseline, network access) are in `global-requirements-intune-solutions.md` — this file adds the **GCC-High-specific** and **device-preparation-specific** requirements on top.

> ***Portal reminder:*** *all verification below uses the `*.us` admin centers — `https://intune.microsoft.us`, `https://entra.microsoft.us` *(verify for GCC High)*, `https://portal.azure.us`, `https://portal.office365.us`. Sign-in is at `https://login.microsoftonline.us`.*

R1. Genuine GCC High (or DoD) tenant — not Commercial/GCC

Description: The tenant is a **GCC High** or **DoD** government-cloud tenant (Azure Government, IL4/IL5), not a Commercial tenant and not "GCC" (which, for Intune, *is* the Commercial service instance). **Why:** The supported technology, portals, and endpoints in this solution only apply to the government cloud. Classic Autopilot's absence — and device preparation's availability — are properties of GCC High/DoD specifically. **Verify:** You sign in at `https://login.microsoftonline.us` and reach `https://intune.microsoft.us`. In Intune, **Devices** → **Windows** → **Enrollment** shows the **Windows Autopilot device preparation** tile and you do **not** see a classic "Windows Autopilot" / "Deployment Profiles" import. `WAPG-Discovery.ps1` reports the Graph environment as **USGov**. **Implement:** If you are on Commercial/GCC, this solution's sovereign steps do not apply — use the Commercial edition

instead. There is **no in-place migration** between Commercial and the government cloud (devices must unenroll and re-enroll). *(Microsoft, Intune Government Service overview.)*

R2. US Government licensing (Gov SKUs)

Description: Users are licensed with **US Government** plans that include Intune and Microsoft Entra ID — e.g. **Microsoft 365 G3/G5 (GCC High)** or standalone **Intune Plan 1 for Government**. **Why:** Device preparation, MDM enrollment, and policy delivery require an Intune-bearing Gov license on the provisioning user. **Verify:** `https://portal.office365.us` → **Users** → **Active users** → the pilot user shows a Gov license with Intune; `https://intune.microsoft.us` → **Tenant administration** → **Tenant status** shows active. **Implement:** Assign the appropriate **Government** SKU (not the Commercial SKU) to each provisioning/pilot user.

R3. Administrative roles for device preparation

Description: The operator holds sufficient Entra/Intune roles: **Intune Administrator** (or the least-privilege `WAPG Device Preparation Administrator` role this solution creates), plus **Global Administrator** available for the one-time service-principal/consent steps. **Why:** Creating device preparation profiles, enrollment settings, and the assigned device group requires elevated rights; the least-privilege role needs the **Enrollment time device membership assignment** permission specifically. **Verify:** `https://entra.microsoft.us` *(verify for GCC High)* → **Roles and administrators** → confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; build the scoped `WAPG Device Preparation Administrator` role in `03-implementation.md` Step 4 for day-to-day operators.

R4. Supported Windows 11 build for device preparation

Description: Target devices run a **device-preparation-supported** Windows 11 build: **Windows 11 24H2 or later**, **or** Windows 11 **23H2/22H2 with KB5035942** (March 2024 build 22621/22631.3374) **or later**. **Why:** Windows Autopilot **device preparation** has a higher OS floor than classic Autopilot. **Windows 10 is not supported** for device preparation. **Verify:** On the device, run `winver`; or in Intune → device → **Hardware**. **Implement:** Re-image/refresh with compliant OEM-optimized Windows 11 media (24H2, or 22H2/23H2 patched to KB5035942 or later). *(Microsoft, Autopilot device preparation overview → Requirements.)*

R5. Device not already registered as a classic Autopilot device

Description: The target device is **not** registered/added as a **Windows Autopilot** device anywhere. **Why:** If a device is registered as a classic Autopilot device, the **Autopilot profile takes precedence** over the device preparation policy and provisioning follows the (absent) classic path. **Verify:** The device does not appear under any Windows Autopilot registration. (In GCC High you generally will not have this surface at all; the risk is a device previously registered in a *Commercial* tenant.) **Implement: Deregister** the device from Windows Autopilot before provisioning via device preparation. *(Microsoft, Autopilot device preparation overview.)*

R6. Intune Provisioning Client service principal present

Description: The **Intune Provisioning Client** first-party service principal (**AppId** `f1346770-5b25-470b-88bd-d5744ab7952c`, sometimes named **Intune Autopilot**

ConfidentialClient) exists in the tenant and can be added as a **group owner**. **Why:** Enrollment Time Grouping works by this principal **owning** the assigned device group. Without it, the device group stays empty at OOBE and nothing deploys. **Verify:** `WAPG-Discovery.ps1` reports the principal as **FOUND**; or search for the AppId under **Enterprise applications** in `https://entra.microsoft.us`. **Implement:** Open `https://intune.microsoft.us` once as **Global Administrator** to provision the principal; re-run `WAPG-Discovery.ps1` to confirm.

R7. Users may join devices to Microsoft Entra + automatic enrollment enabled

Description: Provisioning users are permitted to **Entra-join** devices, and **automatic MDM enrollment** is enabled for their scope. **Why:** The user-driven flow Entra-joins the device and auto-enrolls it into Intune during OOBE. **Verify:** `https://entra.microsoft.us` *(verify for GCC High)* → **Devices** → **Device settings** → **Users may join devices to Microsoft Entra** includes your provisioning users; `https://intune.microsoft.us` → **Devices** → **Enrollment** → **Automatic Enrollment** → **MDM user scope** covers them. **Implement:** Set both in `03-implementation.md` Step 3.

R8. Microsoft Graph PowerShell SDK with US Gov environment

Description: The **Microsoft.Graph** PowerShell SDK is installed on the operator workstation and can connect with `-Environment USGov` (or `USGovDoD`). **Why:** All five scripts in this solution connect to **US Gov Graph** (`graph.microsoft.us`); a Commercial/Global connection queries the wrong cloud. **Verify:** `Get-Module -ListAvailable Microsoft.Graph`; then `Connect-MgGraph -Environment USGov` and confirm `(Get-MgContext).Environment` prints **USGov**. **Implement:** `Install-Module Microsoft.Graph -Scope CurrentUser`. *(Microsoft, Microsoft Graph national cloud deployments.)*

R9. Sovereign endpoints reachable; no SSL inspection on the MDM service

Description: The provisioning network reaches the `*.us` **sovereign endpoints** and exempts the Intune MDM service from SSL inspection. **Why:** Enrollment, policy sync, authentication, and the device-preparation deployment service all depend on these endpoints; `*.manage.microsoft.us` must **not** be SSL-inspected. **Verify:** From the provisioning network, resolve/reach `login.microsoftonline.us`, `portal.manage.microsoft.us`, `graph.microsoft.us`, `enterpriseregistration.microsoftonline.us`, and `login.live.com`; confirm NTP (UDP 123) to `time.windows.com`; confirm `*.manage.microsoft.us` is inspection-exempt. **Implement:** Allow Microsoft's published **US Government** URL/IP ranges through the proxy/firewall and exempt `*.manage.microsoft.us` from TLS inspection. Do **not** paste Commercial `*.manage.microsoft.com` values. *(Microsoft, Network endpoints for US government deployments; Autopilot device preparation requirements → Networking.)*

R10. (Conditional) Corporate identifiers, only if blocking personal devices

Description: If you use **enrollment restrictions** to block personal Windows enrollment, the trusted devices' **corporate identifiers** (manufacturer/model/serial) are pre-uploaded. **Why:** Device preparation only needs corporate identifiers when enrollment restrictions would otherwise block the device — it is **not** the classic Autopilot hardware-hash import. **Verify:** `https://intune.microsoft.us` → **Devices** → **Enrollment** → **Corporate device identifiers** lists the trusted devices. **Implement:** Upload identifiers per

`03-implementation.md` Step 11 (only if you block personal devices). *(Microsoft, Autopilot device preparation overview → Corporate identifiers.)*

Assess gate (pass/fail summary)

Proceed to **Configure** only when **all** are TRUE:

- R1 — Confirmed **GCC High/DoD** (Graph **USGov**; device-preparation tile present; no classic Autopilot blade).
- R2 — Provisioning/pilot users hold **Gov** Intune licensing.
- R3 — Operator has **Intune Administrator** (Global Admin available for one-time steps).
- R4 — Devices run a **device-preparation-supported Windows 11** build (not Windows 10).
- R5 — Target devices are **not** registered as classic Autopilot devices.
- R6 — **Intune Provisioning Client** principal exists and can own a group.
- R7 — Users may **Entra-join**; **automatic enrollment** covers them.
- R8 — **Microsoft.Graph** SDK installed; `-Environment USGov` connects.
- R9 — Sovereign endpoints reachable; `*.manage.microsoft.us` **not** SSL-inspected.
- R10 — (If blocking personal devices) corporate identifiers uploaded.

Sources

- Microsoft, *Microsoft Intune Government Service overview* (GCC High/DoD availability; no Commercial↔Gov migration).
- Microsoft, *Overview / requirements of Windows Autopilot device preparation* (Windows 11 build floor, Entra-join only, Provisioning Client owner, corporate identifiers, Autopilot precedence).
- Microsoft, *Network endpoints for US government deployments* (sovereign endpoints, no SSL inspection).
- Microsoft, *Microsoft Graph national cloud deployments* (`-Environment USGov`).

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

Implementation Guide — Windows Autopilot (Government / GCC High)

This guide builds the solution **step by step in the GCC High portals**, entirely on the **supported** technology — **Windows Autopilot device preparation** (user-driven, Microsoft Entra join). It assumes the

Assess gate in `02-requirements.md` passed. Object names are used **verbatim** from `01-solution-overview.md` — reuse them exactly. GCC-High-only caveats appear **inline at the step where they matter**, boxed like this:

> ***GCC High caveat:*** *…*

Portals used (bookmark these):

- Intune admin center: `https://intune.microsoft.us`
- Entra admin center: `https://entra.microsoft.us` *(verify for GCC High)*
- Azure portal: `https://portal.azure.us`
- Microsoft 365 admin: `https://portal.office365.us`
- Sign-in: `https://login.microsoftonline.us`

> ***GCC High caveat (applies to the whole guide):*** *Every navigation path below is written for the* `*.us` *portals. If a screen or blade shown in a Commercial Autopilot tutorial is missing here, it is almost always because the feature* ***is not available in GCC High*** *(see the parity table in* `01-solution-overview.md` *and* `gcc-high-parity-notes.md`*) — it is* ***not a bug****. In particular,* ***do not*** *look for* ***"Windows Autopilot" / "Deployment Profiles" / "Windows Autopilot devices" (hardware-hash import)****:* ***classic Autopilot does not exist in GCC High.*** *Use* ***Windows Autopilot device preparation*** *throughout.*

Phase A — Configure (build the objects)

Step 1 — Create the Microsoft Entra security groups

Sign in to `https://entra.microsoft.us` *(verify for GCC High)* as Intune Administrator.

1. **Groups** → **All groups** → **New group.**
2. Create `WAPG-Provisioning-Users`

- **Group type:** Security · **Membership type:** Assigned
- Add the pilot users who will provision devices. Select **Create**.

1. **New group** again → create `WAPG-Pilot`

- **Group type:** Security · **Membership type:** Assigned
- Add the same pilot users (this group targets policies/apps to the pilot). **Create.**

1. **New group** again → create `WAPG-Autopilot-Devices`

- **Group type:** Security
- **Membership type: Assigned** — **not** Dynamic.

> ***GCC High caveat:*** *The device group* ***must*** *be* ***Assigned****. Enrollment Time Grouping (how devices land in the group during OOBE) works by the provisioning service principal *owning* an assigned group; a* ***dynamic*** *group breaks it.*

- Do **not** add members. Select **Create**.

Step 2 — Add the Intune Provisioning Client as owner of the device group

This is the step most often missed, and it is the single reason "nothing deploys during OOBE."

1. Still in `https://entra.microsoft.us` → **Groups** → **All groups** →

`WAPG-Autopilot-Devices` → **Owners** → **Add owners**.

1. Search for **Intune Provisioning Client**.

> ***GCC High caveat:*** *In some tenants this principal is named* ***Intune Autopilot ConfidentialClient****. Match by* ***AppId*** `f1346770-5b25-470b-88bd-d5744ab7952c`*, not by name. If it is not present at all, open* `https://intune.microsoft.us` *once as Global Administrator to provision it (R6), then return.*

1. Select it and **Add**. Confirm it now appears under **Owners**.

Run `WAPG-Validation.ps1` later to confirm the ownership programmatically against US Gov Graph.

Step 3 — Allow users to join devices to Entra + enable automatic enrollment

1. `https://entra.microsoft.us` *(verify for GCC High)* → **Devices** → **Device settings** →

Users may join devices to Microsoft Entra → set **Selected** and add `WAPG-Provisioning-Users` (or **All**). **Save.**

1. `https://intune.microsoft.us` → **Devices** → **Enrollment** → **Automatic Enrollment** → set

MDM user scope to **Some** and select `WAPG-Provisioning-Users` (or **All**). Leave the MDM terms/discovery URLs at default. **Save.**

> ***GCC High caveat:*** *The enrollment/discovery URLs default to the sovereign* `*.manage.microsoft.us` *service — do* ***not*** *paste Commercial* `*.manage.microsoft.com` *values.*

Step 4 — Create the least-privilege RBAC role `WAPG Device Preparation Administrator`

1. `https://intune.microsoft.us` → **Tenant administration** → **Roles** → **All roles** →

Create → **Intune role**.

1. **Basics:** Name = `WAPG Device Preparation Administrator`; add a description.
2. **Permissions** — set exactly (all others **No**):

• **Device configurations:** Read = Yes, Create = Yes, Update = Yes, Delete = Yes, Assign = Yes

• **Enrollment programs: Enrollment time device membership assignment = Yes**

• **Managed apps:** Read = Yes

• **Mobile apps:** Read = Yes

• **Organization:** Read = Yes

1. **Scope tags:** add `WAPG` (create it if prompted). **Next** → **Create.**
2. **Assignments** → **Assign** → members = your admin group; scope (groups) = `WAPG-Pilot`; scope tags = `WAPG`.

(Permission set: Microsoft, *Autopilot device preparation requirements → RBAC*.)

Step 5 — Create the Enrollment Status Page (ESP) profile

1. `https://intune.microsoft.us` → **Devices → Enrollment → Enrollment Status Page → Create**.

1. Name = `WAPG-ESP-Provisioning`.

2. **Settings:**

• **Show app and profile configuration progress** = **Yes**

• **Block device use until all apps and profiles are installed** = **Yes**

• Set a **timeout** (e.g. 60 minutes) and allow reset/retry.

• **Only show page to devices provisioned by out-of-box experience (OOBE)** = **Yes**

• Optionally require selected apps to install before desktop access (keep the list **short**).

> ***GCC High caveat:*** *Keep the OOBE-blocking app count small. Microsoft deliberately limits how many apps run during device-preparation OOBE "to increase stability and achieve a higher success rate." Heavy app lists reduce success rates.*

1. **Assignments:** assign to `WAPG-Pilot`. **Create.**

Step 6 — Create a Windows update ring (the GCC High servicing mechanism)

> ***GCC High caveat:*** *Do **not** create Feature/Quality/Expedite/Driver update policies or plan **Windows Autopatch — none are available in GCC High.** Use **Update rings**.*

1. `https://intune.microsoft.us` → **Devices → Windows → Update rings for Windows 10 and later → Create profile**.

1. Name = `WAPG-UpdateRing-Pilot`. Configure a conservative pilot ring (e.g. quality deferral 0–7 days, feature deferral per your policy, active hours).

1. **Assignments:** assign to `WAPG-Pilot`. **Create.**

Step 7 — Create a compliance policy

1. `https://intune.microsoft.us` → **Devices → Compliance → Create policy** → Platform = **Windows 10 and later**.

1. Name = `WAPG-Compliance-Win11`. Set baseline rules (e.g. BitLocker required, Secure Boot, minimum OS version matching R4, firewall on, Defender healthy).

> ***GCC High caveat:*** *Do **not** rely on a compliance rule that requires **Device Health Attestation** results — DHA is **not available in GCC High**. Use settings that evaluate **locally** (BitLocker, Secure Boot, OS build, Defender status).*

1. **Assignments:** assign to `WAPG-Pilot`. **Create.**

Step 8 — (Optional) Windows Hello for Business policy

1. `https://intune.microsoft.us` → **Devices → Enrollment → Windows → Windows Hello for Business**, or a device-configuration WHfB profile targeted to `WAPG-Pilot`.

1. Configure PIN/biometric per your standard.

> ***GCC High caveat:*** *WHfB provisioning works, but the* ***Microsoft hosted PIN reset service is not available in Azure Government****. Plan PIN recovery via allowed sign-in URLs (Intune/CSP/GPO) rather than the hosted PIN-reset service. *(Verify for GCC High for your exact Windows build.)**

Step 9 — Add the applications/scripts to deploy during OOBE

1. `https://intune.microsoft.us` → **Apps** → **Windows** → **Add** — add your Win32 / line-of-business apps and **assign** them to `WAPG-Autopilot-Devices`.

1. **Devices** → **Scripts and remediations** (or **Devices** → **Windows** → **PowerShell scripts**) — add any provisioning PowerShell scripts and assign to `WAPG-Autopilot-Devices`.

> ***GCC High caveat:*** *Do* ***not*** *plan* ***Microsoft Store for Business*** *app provisioning (not available in GCC High) and do* ***not*** *rely on* ***Delivery Optimization for Win32 apps*** *peer caching (not available). Apps download from the cloud per device.*

Only the apps/scripts you **also select inside the device preparation profile** (next step) run during OOBE; anything else assigned to the group installs **after** provisioning completes.

Step 10 — Create the Autopilot device preparation profile

> ***This replaces classic Autopilot deployment profiles****, which do not exist in GCC High.*

1. `https://intune.microsoft.us` → **Devices** → **Windows** → **Enrollment** → tile **Windows Autopilot device preparation → Device preparation policies → Create**.

(If you do not see this tile, re-confirm R1 — you may be on a Commercial/GCC tenant.)

1. **Basics:** Name = `WAPG-DevicePrep-UserDriven-EntraJoin`. **Next.**

2. **Configuration settings:**

• **Deployment settings** → **Deployment mode: User-driven**

> ***GCC High caveat:*** *Choose* ***User-driven****.* ***Self-deploying*** *and* ***pre-provisioning*** *(white-glove) modes are* ***not yet supported in GCC High*** *(planned).*

• **Join type: Microsoft Entra join** (the only supported type — **Hybrid Entra join is not available** in device preparation).

• **Device group:** select `WAPG-Autopilot-Devices` (the assigned group owned by the Intune Provisioning Client from Step 2).

• **Deployment settings** → **Allowed apps / scripts:** select the specific apps and PowerShell scripts (from Step 9) to run **during OOBE**. Keep the list short.

• Leave **device rename during OOBE** unset.

> ***GCC High caveat:*** *Renaming the device during OOBE based on org structure is* ***not yet supported in GCC High*** *(planned). Rename post-provisioning if needed.*

1. **Assignments:** assign the profile to `WAPG-Provisioning-Users` (device preparation profiles target the **users** who provision, not a device group). **Next** → **Create.**

Step 11 — (Optional) Corporate identifiers, only if blocking personal devices

If you use **enrollment restrictions** to block personal Windows enrollment, pre-upload trusted devices so they can still provision:

1. `https://intune.microsoft.us` → **Devices → Enrollment → Corporate device identifiers →

Add** → upload manufacturer/model/serial for the trusted devices.

> ***GCC High caveat:*** *This is **not** the classic Autopilot hardware-hash import (which does not exist here). Corporate identifiers are only needed when enrollment restrictions would otherwise block the device.*

Phase B — Pilot (provision real devices)

1. Confirm a pilot user is in `WAPG-Provisioning-Users` and `WAPG-Pilot`, is

Gov-licensed (R2), and may join devices (R7).

1. On a supported Windows 11 device (R4), **factory-fresh or reset**, begin OOBE:

• Connect to a network that reaches the sovereign endpoints (R9).

• At sign-in, enter the pilot user's **GCC High** account and authenticate at

`https://login.microsoftonline.us`.

1. The device Entra-joins, MDM-enrolls, is added to `WAPG-Autopilot-Devices` **at enrollment

time**, shows the `WAPG-ESP-Provisioning` progress percentage, installs the selected apps/scripts, applies `WAPG-Compliance-Win11` and `WAPG-UpdateRing-Pilot`, and lands on the desktop.

> ***GCC High caveat:*** *If OOBE finishes but the group is empty and nothing installed, the Intune Provisioning Client owner (Step 2) is almost certainly missing or wrong — see `06-troubleshooting.md`, symptom #1.*

Proceed to `05-testing.md` for the **Validate** gate, then `WAPG-Set-ScopeTargeting.ps1` (default `-WhatIf`) to **Expand**.

Sources

• Microsoft, *Overview / requirements of Windows Autopilot device preparation* (Entra-join

only, user-driven only in GCC High, assigned device group, Intune Provisioning Client owner, RBAC permission set, OOBE app limit, corporate identifiers).

• Microsoft, *Intune Government Service overview* (features not available/planned in GCC High:

classic Autopilot, Autopatch/update policies, DFCI, DHA, Store for Business, self-deploying/pre-provisioning, OOBE rename).

• Microsoft, *Network endpoints for US government deployments* (portals, `*.us` endpoints).

Testing & Validation — Windows Autopilot (Government / GCC High)

This is the **Validate** gate. Every test has an explicit **pass/fail** condition. Do not **Expand** to more groups until all gates pass. Read-only scripts (`WAPG-Validation.ps1`, `WAPG-Discovery.ps1`, `WAPG-Report.ps1`) automate much of this against **US Gov Graph** (`-Environment USGov`).

> ***GCC High reminder:*** *All portal checks use the* `*.us` *admin centers (*`https://intune.microsoft.us`, `https://entra.microsoft.us` **(verify for GCC High)*).*

T1 — Configuration objects exist and are correct (pre-pilot)

Run: `WAPG-Validation.ps1` (read-only).

Check	Pass
Groups `WAPG-Provisioning-Users`, `WAPG-Pilot`, `WAPG-Autopilot-Devices` exist	All three present
`WAPG-Autopilot-Devices` is **Assigned** (not Dynamic)	membershipType = Assigned
Intune Provisioning Client (`f1346770-…`) is an **owner** of `WAPG-Autopilot-Devices`	Owner present
Device preparation profile `WAPG-DevicePrep-UserDriven-EntraJoin` exists, **User-driven**, **Entra join**	Confirmed
ESP `WAPG-ESP-Provisioning` assigned to `WAPG-Pilot`	Assigned
Update ring `WAPG-UpdateRing-Pilot` + compliance `WAPG-Compliance-Win11` assigned	Assigned

Fail → fix the missing/incorrect object in `03-implementation.md` before piloting. The owner check is the highest-value gate.

T2 — Tenant is genuinely GCC High (not Commercial)

Check: Signed into `https://intune.microsoft.us` you can see the **Windows Autopilot device preparation** tile, and you do **not** see a classic "Windows Autopilot" / "Deployment Profiles" import. `WAPG-Discovery.ps1` reports the connected Graph environment as **USGov**. **Pass:** Graph environment = USGov; device-preparation tile present. **Fail:** If you see classic Autopilot blades or Graph reports Global/Commercial, you are on the wrong tenant/cloud — stop and re-confirm R1.

T3 — Endpoint reachability from the provisioning network

Check: From the network a pilot device will use, resolve and reach: `login.microsoftonline.us`, `portal.manage.microsoft.us`, `graph.microsoft.us`, `enterpriseregistration.microsoftonline.us`, `login.live.com`, and NTP `time.windows.com` (UDP 123). Confirm **no SSL inspection** on `*.manage.microsoft.us`. **Pass:** All resolve and connect; `*.manage.microsoft.us` is inspection-exempt. **Fail:** Any blocked/inspected → fix per R9 before piloting.

T4 — End-to-end pilot provisioning (the core test)

Run: provision one factory-fresh/reset supported Windows 11 device with a pilot user (per `03-implementation.md` Phase B).

Gate	Pass
Device Entra-joins during OOBE	Joined (visible later in Entra → Devices)
Device auto-enrolls into Intune	Appears in Intune → Devices → All devices
Device is added to `WAPG-Autopilot-Devices` at enrollment	Member appears automatically
ESP `WAPG-ESP-Provisioning` shows progress percentage	Percentage UI shown
Selected OOBE apps/scripts install	Status = installed/succeeded
User reaches desktop without technician touch	Zero-touch achieved
Device shows **compliant** against `WAPG-Compliance-Win11`	Compliant (allow grace period)

Fail on empty group / no installs → Intune Provisioning Client owner missing (`06-troubleshooting.md`, symptom #1).

T5 — Deployment monitoring reflects the run

Check: `https://intune.microsoft.us` → **Devices** → **Windows** → **Enrollment** → **Windows Autopilot device preparation** → **Monitor** (deployment report). The pilot device appears with profile name/version, deployment status, per-app and per-script status. `WAPG-Report.ps1` produces the same summary from US Gov Graph. **Pass:** The device and its app/script statuses are visible. **Fail:** No record → device likely never matched the profile (user not in `WAPG-Provisioning-Users`, or device pre-registered as classic Autopilot — which takes precedence).

T6 — Servicing and policy land (post-provisioning)

Check: After provisioning, `WAPG-UpdateRing-Pilot` and any post-OOBE apps assigned to `WAPG-Autopilot-Devices` apply within a normal sync window. **Pass:** Update ring shows assigned/applied; post-OOBE apps install. **Fail:** Confirm assignments target the device group and the device is a member.

> ***GCC High reminder:*** *Do not test for Autopatch/feature-update-policy behavior — those are* ***not available in GCC High****. Servicing is validated only via the* ***update ring****.*

Validate gate (pass/fail summary)

Proceed to **Expand** only when **all** are TRUE:

- T1 — All objects correct; **Intune Provisioning Client owns the device group.**

- T2 — Confirmed **GCC High** (Graph USGov; device-preparation tile present).
- T3 — Sovereign endpoints reachable; **no SSL inspection** on `*.manage.microsoft.us`.
- T4 — One device provisioned **zero-touch** end to end and is compliant.
- T5 — Deployment **monitoring** shows the device with app/script status.
- T6 — Update ring + post-OOBE policy/apps applied.

Then run `WAPG-Set-ScopeTargeting.ps1` (default `-WhatIf`) to preview widening scope, review, and re-run with `-WhatIf:$false` to apply.

Sources

- Microsoft, *Windows Autopilot device preparation — overview, requirements, reporting and monitoring.*
- Microsoft, *Intune Government Service overview* (GCC High feature availability).
- Microsoft, *Network endpoints for US government deployments.*

Troubleshooting — Windows Autopilot (Government / GCC High)

Symptoms and fixes, ordered by how often they bite on a **GCC High / DoD** tenant. Many are GCC-High-specific (wrong cloud, missing sovereign feature, wrong portal). Run `WAPG-Validation.ps1` and `WAPG-Discovery.ps1` (read-only, `-Environment USGov`) to gather evidence before changing anything.

> ***Portal reminder:*** *troubleshoot in the* `*.us` *portals —* `https://intune.microsoft.us`*,* `https://entra.microsoft.us` **(verify for GCC High)*,* `https://portal.azure.us`*.*

1. OOBE completes but the device group is empty and nothing installed

By far the most common failure. The device Entra-joins and enrolls, but no apps/scripts run and `WAPG-Autopilot-Devices` has no members.

Cause: The **Intune Provisioning Client** service principal (`f1346770-5b25-470b-88bd-d5744ab7952c`) is **not an owner** of the assigned device group, so **Enrollment Time Grouping** can't add the device. **Fix:**

1. `WAPG-Validation.ps1` → confirm the "Provisioning Client OWNS …" check is **FAIL**.
2. `WAPG-Remediation.ps1` (preview), then `-WhatIf:$false` to add the owner — **or** `entra.microsoft.us` → **Groups** → `WAPG-Autopilot-Devices` → **Owners** → **Add owners** → add the principal (match by **AppId**, may be named *Intune Autopilot ConfidentialClient*).
1. Reset the device and re-provision.

2. The device group is Dynamic, not Assigned

Cause: `WAPG-Autopilot-Devices` was created as a **Dynamic** group; Enrollment Time Grouping requires an **Assigned** group and the owner mechanism does not apply. **Fix:** Recreate the group as **Assigned** (`03-implementation.md` Step 1), re-add the Provisioning Client owner (Step 2), point the device preparation profile at it, re-provision.

3. "I don't see Windows Autopilot / Deployment Profiles / hardware-hash import"

Cause: This is **expected** — **classic Windows Autopilot is not available in GCC High/DoD** (and there is no plan to add it). You are looking for a Commercial-only surface. **Fix:** Use **Windows Autopilot device preparation** instead (`intune.microsoft.us` → **Devices** → **Windows** → **Enrollment** → **Windows Autopilot device preparation**). Do not import hardware hashes; the user-driven flow does not need them. (*Confirmed: Microsoft, Intune Government Service overview.*)

4. Scripts return nothing / see the "wrong" tenant

Cause: The Graph session connected to the **Commercial** cloud, not US Gov. **Fix:** Always pass `-Environment USGov` (or `USGovDoD`). Every script prints `Environment = …` on connect — confirm it says **USGov**. If it says Global, disconnect (`Disconnect-MgGraph`) and reconnect with the correct environment. The scripts refuse to make changes when the environment isn't US Gov.

5. Provisioning fails at sign-in / can't reach services

Cause: Sovereign endpoints blocked, or **SSL inspection** on `*.manage.microsoft.us`. **Fix:** Allow (and exempt from SSL inspection) `*.manage.microsoft.us`, `login.microsoftonline.us`, `enterpriseregistration.microsoftonline.us`, `graph.microsoft.us`, `login.live.com`, `*.msftconnecttest.com`, `*.events.data.microsoft.com`, and NTP UDP 123 to `time.windows.com` (R9). Do **not** paste Commercial `*.manage.microsoft.com` values.

6. Device won't provision — unsupported Windows build

Cause: Device runs **Windows 10** or a Windows 11 build without **KB5035942**. **Fix:** Device preparation requires **Windows 11 24H2+** or **22H2/23H2 with KB5035942** (March 2024 media or later). Re-image/refresh with compliant OEM-optimized media (R4). **Windows 10 is not supported** for device preparation.

7. Self-deploying / pre-provisioning / Hybrid join / device-rename options are missing

Cause: Those modes are **not available in GCC High.** Self-deploying, pre-provisioning (white-glove), and OOBE device-rename are listed as **planned**; **Hybrid Microsoft Entra join is not supported** — device preparation is **Entra join only**. **Fix:** Use **user-driven** deployment with **Microsoft Entra join**; rename devices **after** provisioning. Do not design kiosk/self-deploying or hybrid-join flows on GCC High. (*Confirmed: Intune Government Service overview; Autopilot device preparation overview.*)

8. Too many apps during OOBE / low provisioning success rate

Cause: Large OOBE app list — Microsoft deliberately limits apps during device-prep OOBE for stability. **Fix:** In `WAPG-DevicePrep-UserDriven-EntraJoin` select only the **essential** apps/scripts for OOBE; assign the rest to `WAPG-Autopilot-Devices` for post-provisioning install. Keep the ESP blocking app list short.

9. Windows Hello for Business PIN reset doesn't work

Cause: The **Microsoft hosted PIN reset service is not available in Azure Government. Fix:** Configure PIN recovery via allowed sign-in URLs through Intune/CSP/GPO rather than the hosted service. Validate on a pilot device. *(Verify for GCC High for your exact Windows build.)*

10. Update/servicing policies don't appear or don't apply

Cause: You configured **Windows Autopatch** or **Feature/Quality/Expedite/Driver update policies** — **none are available in GCC High. Fix:** Service via **Update rings for Windows 10 and later** (`WAPG-UpdateRing-Pilot`). Remove any dependency on Autopatch or update-policy reports.

11. Compliance policy always evaluates non-compliant on an attestation rule

Cause: A rule requires **Device Health Attestation** results — **DHA is not available in GCC High. Fix:** Use locally evaluated rules (BitLocker, Secure Boot, OS build, Defender) in `WAPG-Compliance-Win11`; remove DHA-dependent rules.

12. Certificate provisioning expected via Cloud PKI

Cause: Cloud PKI is not available in GCC High. Fix: Use **SCEP/PKCS** with your own PKI/NDES for certificate delivery instead.

13. Remote assistance for a stuck device isn't available

Cause: Remote Help is not available in GCC High (planned). **Fix:** Use your organization's approved alternative remote-assist tool for post-provisioning support (document it in your runbook).

14. Device was previously a classic Autopilot device elsewhere

Cause: If a device is registered/added as a **Windows Autopilot** device, that profile **takes precedence** over the device preparation policy. **Fix:** Deregister the device from Windows Autopilot, then re-provision via device preparation. (*Confirmed: Autopilot device preparation overview.*)

15. RBAC admin can't configure device preparation

Cause: The role lacks **Enrollment programs** → **Enrollment time device membership assignment** (or the Device configurations set). **Fix:** Confirm `WAPG Device Preparation Administrator` has exactly the permissions in `03-implementation.md` Step 4, including **Enrollment time device membership assignment = Yes**.

Escalation checklist (attach to any support case)

- Output of `WAPG-Discovery.ps1` and `WAPG-Validation.ps1` (confirming **USGov**).
- The device group's **membership type** (Assigned) and **Owners** list.
- The device preparation profile mode (**user-driven**) and **join type** (**Entra join**).
- Confirmation the tenant is **GCC High/DoD** (not Commercial/GCC).
- Endpoint/proxy config showing sovereign URLs allowed and `*.manage.microsoft.us` exempt from

SSL inspection.

- The Windows build on the failing device (`winver`).

Sources

- Microsoft, *Intune Government Service overview* (not-available/planned features).
- Microsoft, *Overview / requirements of Windows Autopilot device preparation* (Assigned group +

Provisioning Client owner, Entra-join only, OOBE app limit, Autopilot precedence, Windows 11 build floor).

- Microsoft, *Network endpoints for US government deployments* (sovereign endpoints, no SSL

inspection).

- Microsoft, *Configure Windows Hello for Business* (PIN reset service unavailable in Azure

Government).

Best Practices — Windows Autopilot (Government / GCC High)

Operating guidance for running Windows Autopilot **device preparation** correctly on a **GCC High / DoD** tenant. These are the habits that keep a sovereign deployment stable and keep you from accidentally designing against Commercial-only features.

> ***Portal reminder:*** *operate in the* `*.us` *portals (*`https://intune.microsoft.us`*,* `https://entra.microsoft.us` **(verify for GCC High)*). Connect PowerShell with* `-Environment USGov` *and confirm the environment prints as* ***USGov*** *before acting.*

1. Design for the supported path only — never for classic Autopilot

Classic Windows Autopilot (hardware-hash import, deployment profiles, self-deploying and pre-provisioning modes, Hybrid Entra join) is **not available in GCC High**. Do not scope, budget, or document a rollout around it. Everything is **device preparation**, **user-driven**, **Entra join**. If a stakeholder asks for white-glove/self-deploying kiosks, set expectations early: those modes are **planned**, not available today.

2. Treat the Provisioning Client owner as the linchpin

The most common — and most confusing — failure is a device that Entra-joins and enrolls but receives **no apps/scripts** because the **Intune Provisioning Client** service principal does not **own** the assigned device group. Make ownership an explicit, verified step:

- The device group (`WAPG-Autopilot-Devices`) is **Assigned**, never Dynamic.
- The Provisioning Client (**AppId** `f1346770-5b25-470b-88bd-d5744ab7952c`, possibly named

Intune Autopilot ConfidentialClient) is an **Owner**.

- Confirm it programmatically with `WAPG-Validation.ps1` (US Gov Graph) before every pilot.

3. Keep the OOBE app/script list short

Microsoft deliberately limits how many apps run during device-preparation OOBE for stability and success rate. Select only the **essential** apps/scripts inside `WAPG-DevicePrep-UserDriven-EntraJoin`; assign everything else to `WAPG-Autopilot-Devices` for **post-provisioning** install. Keep the ESP blocking list short so users reach the desktop quickly.

4. Service Windows through update rings, not Autopatch

Windows Autopatch and the newer Feature/Quality/Expedite/Driver update policies are **not available in GCC High**. Standardize on **Update rings for Windows 10 and later** (`WAPG-UpdateRing-Pilot` for the pilot, then wider rings). Do not design a servicing plan that assumes Autopatch, update-policy reports, or Windows Subscription Activation.

5. Build compliance on locally evaluated signals

Device Health Attestation is not available in GCC High. Base `WAPG-Compliance-Win11` on rules that evaluate on the device — **BitLocker, Secure Boot, minimum OS build, firewall, Defender status** — not attestation results, so devices are not stuck non-compliant.

6. Plan the gaps that touch day-2 support

- **Remote Help** is unavailable — pre-select and document an approved alternative

remote-assist tool for post-provisioning support.

- **Cloud PKI** is unavailable — deliver certificates via **SCEP/PKCS** with your own PKI/NDES.
- **Windows Hello for Business** works, but the **hosted PIN reset service is unavailable in

Azure Government** — configure PIN recovery via allowed sign-in URLs (Intune/CSP/GPO).

- **Microsoft Store for Business** is unavailable — package apps as Win32/line-of-business.

7. Get the network right before the first device

Allow the **US Government** URL/IP ranges and **exempt** `*.manage.microsoft.us` **from SSL inspection**. Device preparation also contacts `login.live.com` for its deployment service and needs NTP (UDP 123). One misconfigured proxy will fail every OOBE identically — validate T3 before piloting a single device.

8. Use the scripts the way they are designed

- **Read-only first:** `WAPG-Discovery.ps1` and `WAPG-Validation.ps1` before any change.
- **Report for evidence:** `WAPG-Report.ps1` for deployment status you can attach to a change record or support case.
- **Remediate safely:** `WAPG-Remediation.ps1` and `WAPG-Set-ScopeTargeting.ps1` default to `-WhatIf` — preview, review, then re-run with `-WhatIf:$false`. Every script refuses to change anything when the connected environment is not **US Gov**.

9. Respect the gates — pilot before you expand

Follow **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**. Widen scope only after the `05-testing.md` gates pass, and widen **one group at a time** with `WAPG-Set-ScopeTargeting.ps1` (preview first). A device that was fine in the pilot can still surface a network or licensing gap in a new business unit — expand deliberately.

10. Keep a "verify for GCC High" watchlist

A few details in `gcc-high-parity-notes.md` are flagged **"verify for GCC High"** because Microsoft's public docs did not confirm them verbatim at authoring time (e.g. the standalone `entra.microsoft.us` host, exact WHfB behavior under device-preparation OOBE). Validate those against your live tenant and update your internal runbook — never assume.

Sources

- Microsoft, *Overview / requirements of Windows Autopilot device preparation* (OOBE app limit, Assigned group + Provisioning Client owner, Entra-join only).
- Microsoft, *Intune Government Service overview* (unavailable/planned features: classic Autopilot, Autopatch/update policies, DHA, Cloud PKI, Store for Business, Remote Help).
- Microsoft, *Network endpoints for US government deployments* (sovereign endpoints, no SSL inspection).
- Microsoft, *Microsoft Graph national cloud deployments* (`-Environment USGov`).

Deliverables Summary — Windows Autopilot (Government / GCC High)

Everything included in this **Government (GCC High) edition** of Windows Autopilot, and how the pieces fit the **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model. The solution is built entirely on the **supported** GCC High technology — **Windows Autopilot device preparation** (user-driven, Microsoft Entra join) — because **classic Windows Autopilot is not available in GCC High/DoD**.

Documents (in `source/`)

File	What it gives you
`01-solution-overview.md`	The pivot (classic Autopilot unavailable → device preparation), the **Commercial vs GCC High — what's different** tables, the named-object set, and the deployment model.
`02-requirements.md`	The **Assess** gate — GCC-High- and device-preparation-specific prerequisites (R1–R10) with Verify/Implement for each, on top of the category `global-requirements`.
`03-implementation.md`	The **Configure** + **Pilot** steps — exact `*.us` portal navigation, named objects, GCC-High caveats inline at each step, culminating in the device preparation profile.
`05-testing.md`	The **Validate** gate — T1–T6 with explicit pass/fail conditions.
`06-troubleshooting.md`	GCC-High-ordered symptoms and fixes, plus an escalation checklist.
`07-best-practices.md`	Operating habits for a stable sovereign deployment.
`gcc-high-parity-notes.md`	The **cited** Commercial-vs-GCC-High reference, separating doc-confirmed facts from **"verify for GCC High"** flags.
`executive-summary.md`	One-page situation/complication/resolution for decision-makers.
`global-requirements-intune-solutions.md`	Category-level prerequisites shared by all Intune solutions.

PowerShell scripts (in `scripts/`)

All five connect to **Microsoft Graph US Gov** (`-Environment USGov`; `graph.microsoft.us`), print the resolved environment on connect, and **refuse to make changes** when the environment is not US Gov. Discovery/Validation/Report are **read-only**; Remediation and Set-ScopeTargeting default to `-WhatIf` (preview before change).

Script	Purpose	Changes anything?
`WAPG-Discovery.ps1`	Read-only inventory of the current GCC High provisioning posture (environment, device-preparation profiles, `WAPG` groups, Provisioning Client SP).	No

Script	Purpose	Changes anything?
`WAPG-Validation.ps1`	Read-only pass/fail checks for the **Validate** gate (groups exist, device group Assigned, **Provisioning Client owns it**, profile mode/join type, ESP/ring/compliance assignments).	No
`WAPG-Report.ps1`	Read-only deployment-status report (per-device / per-app / per-script) from the device-preparation monitoring surface.	No
`WAPG-Remediation.ps1`	Safely fixes the top failure — adds the **Intune Provisioning Client** as **owner** of the assigned device group.	Only with `-WhatIf:$false`
`WAPG-Set-ScopeTargeting.ps1`	Safely widens provisioning scope during **Expand** (adds members to a `WAPG` scope group).	Only with `-WhatIf:$false`

Named objects you will have built

`WAPG-Provisioning-Users` · `WAPG-Pilot` · `WAPG-Autopilot-Devices` (Assigned, Provisioning Client owner) · `WAPG-DevicePrep-UserDriven-EntraJoin` · `WAPG-ESP-Provisioning` · `WAPG Device Preparation Administrator` (RBAC) · `WAPG-UpdateRing-Pilot` · `WAPG-Compliance-Win11` · `WAPG` (scope tag).

Definition of done

- Every **Assess** requirement (R1–R10) verified.
- All **Configure** objects built with the exact names above, in the `*.us` portals.
- One device provisioned **zero-touch** end to end in **Pilot**.
- Every **Validate** gate (T1–T6) passes — most importantly, the Provisioning Client **owns**

the device group.

- Scope widened deliberately in **Expand** with `WAPG-Set-ScopeTargeting.ps1` (preview first).

The one thing to remember

In GCC High there is no classic Autopilot, no hardware hash, and no deployment profile. The supported path is **Windows Autopilot device preparation** — user-driven, Entra join, assigned device group owned by the Intune Provisioning Client. Build for that, service via update rings, and validate on a pilot before you scale.

Sources

- Microsoft, *Intune Government Service overview*; *Windows Autopilot device preparation overview / requirements*; *Network endpoints for US government deployments*; *Microsoft Graph national cloud deployments*. Full cited list in `gcc-high-parity-notes.md`.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

WAPG-Discovery.ps1

Purpose: Connects to Microsoft Graph US Gov and reports the current state relevant to Windows Autopilot on a GCC High / DoD tenant, built on the supported technology (Windows Autopilot device preparation): the tenant/environment, device-preparation profiles, the named WAPG groups, and the Intune Provisioning Client service principal. Read-only. It changes nothing; it only describes what exists so you can plan the build. Placeholders use hyphenated names such as your-tenant-id and your-group-name. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WAPG-Discovery.ps1 (Graph scopes; -Environment USGov) Scopes requested: Directory.Read.All, Group.Read.All, DeviceManagementServiceConfig.Read.All, DeviceManagementConfiguration.Read.All, Application.Read.All EXPECTED RESULTS A printed summary. PASS when the connected environment prints as USGov and the report renders. FAIL (stop and re-check the tenant/cloud) if the environment is not USGov or connection fails. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\WAPG-Discovery.ps1 (Graph scopes; -Environment USGov) Scopes requested: Directory.Read.All, Group.Read.All, DeviceManagementServiceConfig.Read.All, DeviceManagementConfiguration.Read.All, Application.Read.All EXPECTED RESULTS A printed summary. PASS when the connected environment prints as USGov and the report renders. FAIL (stop and re-check the tenant/cloud) if the environment is not USGov or connection fails. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

WAPG-Remediation.ps1

Purpose: Connects to Microsoft Graph US Gov and remediates the top failure of Windows Autopilot device preparation on a GCC High / DoD tenant: the Intune Provisioning Client service principal not being an OWNER of the assigned device group (which leaves the group empty during OOBE so nothing deploys). Runs in preview mode by default (-WhatIf) and only makes a change when you pass -WhatIf:$false. Placeholders use hyphenated names such as your-tenant-id and your-device-group-name. HOW TO RUN Preview (no change): powershell -ExecutionPolicy Bypass -File .\WAPG-Remediation.ps1 Apply the fix: powershell -ExecutionPolicy Bypass -File .\WAPG-Remediation.ps1 -WhatIf:$false (Graph scopes; -Environment USGov) Scopes requested: Group.ReadWrite.All, Application.Read.All, Directory.Read.All EXPECTED RESULTS Preview prints the intended action and makes no change. Apply adds the Provisioning Client as owner and confirms. PASS when ownership is present afterward; re-run WAPG-Validation.ps1 to confirm. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints except the single, explicit owner-add performed only when -WhatIf:$false is passed. #>

How to run: Preview (no change): powershell -ExecutionPolicy Bypass -File .\WAPG-Remediation.ps1 Apply the fix: powershell -ExecutionPolicy Bypass -File .\WAPG-Remediation.ps1 -WhatIf:$false (Graph scopes; -Environment USGov) Scopes requested: Group.ReadWrite.All, Application.Read.All, Directory.Read.All EXPECTED RESULTS Preview prints the intended action and makes no change. Apply adds the Provisioning Client as owner and confirms. PASS when ownership is present afterward; re-run WAPG-Validation.ps1 to confirm. .NOTES White-label — the repeatable way. Read-only against Microsoft

Graph US Gov endpoints except the single, explicit owner-add performed only when -WhatIf:$false is passed. #>

WAPG-Report.ps1

Purpose: Connects to Microsoft Graph US Gov and produces a human-readable and optional CSV report of Windows Autopilot device preparation outcomes on a GCC High / DoD tenant: devices in the WAPG-Autopilot-Devices group, their Entra-join/enrollment/compliance state, and a rollout summary. Read-only. Use it after piloting and during expansion to evidence results. Placeholders use hyphenated names such as your-tenant-id and your-output-folder. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WAPG-Report.ps1 (Graph scopes; -Environment USGov) Optional: -CsvPath your-output-folder\WAPG-report.csv Scopes requested: Group.Read.All, Directory.Read.All, DeviceManagementManagedDevices.Read.All EXPECTED RESULTS A printed rollout summary (counts by state) and per-device table; a CSV if -CsvPath is given. PASS when the report renders against the USGov environment. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\WAPG-Report.ps1 (Graph scopes; -Environment USGov) Optional: -CsvPath your-output-folder\WAPG-report.csv Scopes requested: Group.Read.All, Directory.Read.All, DeviceManagementManagedDevices.Read.All EXPECTED RESULTS A printed rollout summary (counts by state) and per-device table; a CSV if -CsvPath is given. PASS when the report renders against the USGov environment. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

WAPG-Set-ScopeTargeting.ps1

Purpose: Connects to Microsoft Graph US Gov and adds members (users or groups) to the WAPG provisioning scope on a GCC High / DoD tenant during the Expand phase — for example adding a new business-unit group to WAPG-Provisioning-Users so more users can provision devices via Windows Autopilot device preparation. Runs in preview mode by default (-WhatIf) and only changes membership when you pass -WhatIf:$false. Placeholders use hyphenated names such as your-tenant-id, your-target-group-name, and your-member-object-id. HOW TO RUN Preview (no change): powershell -ExecutionPolicy Bypass -File .\WAPG-Set-ScopeTargeting.ps1 -AddMemberObjectIds your-member-object-id Apply: powershell -ExecutionPolicy Bypass -File .\WAPG-Set-ScopeTargeting.ps1 -AddMemberObjectIds your-member-object-id -WhatIf:$false (Graph scopes; -Environment USGov) Scopes requested: Group.ReadWrite.All, Directory.Read.All EXPECTED RESULTS Preview lists the members that would be added and changes nothing. Apply adds them and confirms. PASS when the target group contains the intended members afterward. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints except the explicit membership add performed only when -WhatIf:$false is passed. #>

How to run: Preview (no change): powershell -ExecutionPolicy Bypass -File .\WAPG-Set-ScopeTargeting.ps1 -AddMemberObjectIds your-member-object-id Apply: powershell -ExecutionPolicy Bypass -File .\WAPG-Set-ScopeTargeting.ps1 -AddMemberObjectIds your-member-object-id -WhatIf:$false (Graph scopes; -Environment USGov) Scopes requested: Group.ReadWrite.All, Directory.Read.All EXPECTED RESULTS Preview lists the members that would be added and changes nothing. Apply adds them and confirms. PASS when the target group contains the intended members afterward. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints except the explicit membership add performed only when -WhatIf:$false is

passed. #>

WAPG-Validation.ps1

Purpose: Connects to Microsoft Graph US Gov and validates the Windows Autopilot device preparation configuration on a GCC High / DoD tenant against explicit pass/fail gates: the named WAPG groups exist, the device group is Assigned (not Dynamic), and the Intune Provisioning Client service principal OWNS the device group (the requirement that makes Enrollment Time Grouping work). Read-only. Placeholders use hyphenated names such as your-tenant-id. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WAPG-Validation.ps1 (Graph scopes; -Environment USGov) Scopes requested: Directory.Read.All, Group.Read.All, Application.Read.All, DeviceManagementConfiguration.Read.All EXPECTED RESULTS A PASS/FAIL line per check and an overall result. Overall PASS means the build is ready to pilot. Any FAIL prints the fix location in 03-implementation.md. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\WAPG-Validation.ps1 (Graph scopes; -Environment USGov) Scopes requested: Directory.Read.All, Group.Read.All, Application.Read.All, DeviceManagementConfiguration.Read.All EXPECTED RESULTS A PASS/FAIL line per check and an overall result. Overall PASS means the build is ready to pilot. Any FAIL prints the fix location in 03-implementation.md. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>